# TP 5 – Analyser un site vulnérable

Le but de ce TP est d'analyser un site vulnérable et de fournir un rapport d'audit et de propositions de modifications.

## I- Mise en place

1) Créez une base de données MySQL nommée par exemple : tp_insecure.

2) Exécutez le script SQL suivant :

```
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    email VARCHAR(255),
    password VARCHAR(255)
);
```

3) Copiez les fichiers fournis dans un dossier accessible par votre serveur web

(par exemple : htdocs/tp_insecure ou www/tp_insecure).

4) Ouvrez le fichier db.php et, si nécessaire, adaptez :

- le nom de la base,
- le login et le mot de passe MySQL.

5) Vérifiez que vous pouvez accéder à la page register.php depuis votre navigateur.

## II Découverte fonctionnelle

1) Testez le scénario suivant :

a) Inscription d'un nouvel utilisateur (register.php).

b) Connexion avec cet utilisateur (login.php).

c) Saisie du code de "double authentification" (verify_2fa.php).

d) Accès à la page dite "sécurisée" (secure.php).

e) Déconnexion (logout.php).

2) Prenez des notes sur :

- Ce que s’affiche à l’écran après l’inscription.
- Ce que vous voyez dans la barre d’adresse (URL) après l’inscription.
- Ce qui se passe si vous entrez un mauvais mot de passe à la connexion.
- Ce qui se passe si vous :
  - arrivez sur verify_2fa.php sans passer par login.php,
  - essayez d’ouvrir directement secure.php dans le navigateur.

3) Dans votre rapport, décrivez en quelques lignes :

- le fonctionnement général du site,
- les comportements qui vous semblent “bizarres” ou suspects.

## III Analyse technique guidée

Ouvrez les fichiers PHP dans un éditeur de code et répondez aux questions.

### III-1 Mots de passe et formulaires

Fichiers concernés : register.php, login.php, secure.php, base de données.

1) Comment les mots de passe sont-ils stockés dans la base de données ?

- Donnez un exemple concret (copie d’écran ou description du contenu de la table users).

2) Après l’inscription, que voyez-vous à l’écran concernant le mot de passe ?

- Est-ce acceptable ? Pourquoi ?

3) Quel est le type de méthode utilisé dans le formulaire d’inscription (GET ou POST) ?

- Observez l’URL après avoir soumis le formulaire. Que constatez-vous ?
- Quels sont les risques ?

4) Y a-t-il des contrôles sur :

- la longueur du mot de passe ?
- la complexité (chiffres, lettres, caractères spéciaux, etc.) ?

5) Dans votre rapport, listez au moins 3 problèmes liés à la gestion des mots de passe et expliquez pourquoi ils sont dangereux.

### III.2 Base de données et requêtes SQL

Fichier concerné : login.php (partie vérification email/mot de passe).

1) Repérez la requête SQL qui sert à vérifier l'email et le mot de passe.

- Comment est-elle construite ? (variables directement concaténées, requête préparée, etc.)

2) Expliquez en quoi cette construction peut être vulnérable aux injections SQL.

3) Essayez de modifier le champ "email" ou "password" lors de la connexion avec une valeur suspecte pour tenter de contourner l'authentification.

Exemple de réflexion : que se passe-t-il si on insère des apostrophes ou des morceaux de condition SQL ?

4) Dans votre rapport :

- décrivez un exemple d'injection possible (même théorique),
- expliquez quel serait l'impact pour le site.

### III.3 Sessions et gestion de la connexion

Fichiers concernés : login.php, verify_2fa.php, secure.php, logout.php.

1) Quelles variables de session sont utilisées pour représenter l'utilisateur connecté ?

- Notez les noms de ces variables (par ex. $_SESSION['xxx']).

2) Dans secure.php :

- Quelle condition est testée pour autoriser ou non l'accès à la page ?
- Cette condition vous semble-t-elle suffisante ? Pourquoi ?

3) Dans verify_2fa.php :

- Quelles variables de session sont utilisées pour la double authentification ?

4) Y a-t-il un renouvellement d'identifiant de session (session_regenerate_id) après connexion ou après validation du code 2FA ?

5) Dans votre rapport, listez au moins 2 faiblesses dans la gestion des sessions.

**III.4 Double authentification (2FA)**

Fichiers concernés : login.php, verify_2fa.php, secure.php.

1) Comment le code 2FA est-il généré ?

- Est-il fixe ou aléatoire ?
- Est-il spécifique à chaque utilisateur ?

2) Où est stocké ce code (session, base, variable simple…) ?

3) Le code a-t-il une durée de validité limitée dans le temps ?

- Si non, quels problèmes cela pose ?

4) Que se passe-t-il si vous :

- cliquez sur "Aller quand même à la page sécurisée" sans entrer de code ?
- tapez directement l'URL secure.php dans le navigateur ?

5) Dans votre rapport, expliquez pourquoi, dans l'état actuel, on ne peut pas parler de véritable "double authentification".

**III.5 Autres aspects de sécurité (XSS, fuite d'informations…)**

Fichiers concernés : db.php, secure.php, éventuellement les autres.

1) Dans db.php, que se passe-t-il en cas d'erreur de connexion à la base ?

- Quelles informations techniques sont montrées à l'écran ?
- Pourquoi est-ce problématique en production ?

2) Dans secure.php, comment l'email de l'utilisateur est-il affiché ?

- Est-ce qu'on utilise une fonction d'échappement (htmlspecialchars, etc.) ?
- Que se passerait-il si l'email enregistré contenait du code HTML ou JavaScript ?

3) Identifiez au moins 1 autre point qui pourrait fuiter des informations techniques ou permettre des attaques côté navigateur.

## IV- Propositions de corrections

Pour chaque catégorie ci-dessus, proposez des corrections.

1) Mots de passe :

- Comment faudrait-il les stocker ? (nom de fonction PHP, principe)
- Quelle méthode utiliser pour le formulaire (GET ou POST) ? Pourquoi ?
- Quelles règles simples pourriez-vous imposer sur la longueur/complexité ?

2) Requêtes SQL :

- Quel mécanisme PHP utiliser pour éviter les injections SQL (mot-clé : “requêtes préparées”) ?

3) Sessions :

- Quelles vérifications supplémentaires ajouter dans secure.php ?
- À quel moment appeler session_regenerate_id(), et pourquoi ?

4) Double authentification :

- Comment faire pour que le code 2FA soit réellement utile ? (code aléatoire, expiration, non affiché en clair, etc.)

5) Autres :

- Quelles informations techniques ne devraient jamais être affichées à l’utilisateur final ?
- Quelle fonction utiliser systématiquement pour afficher des données utilisateurs dans une page HTML ?